Buổi 9

Ngăn xếp (Stack) và Hàng đợi (Queue)

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Danh sách hạn chế

1.1 Khái niệm danh sách hạn chế

2. Ngăn xếp (Stack)

2.1 Các phép toán trên ngăn xếp
2.2 Cài đặt ngăn xếp bằng mảng
2.3 Cài đặt ngăn xếp bằng danh sách liên kết đơn
2.4 Ứng dụng 1: đảo chuỗi và kiểm tra dấu ngoặc
2.5 Minh họa kiểm tra dấu ngoặc
2.6 Ứng dụng 2: chuyển biểu thức trung tố sang hậu tố
2.7 Cài đặt chuyển trung tố sang hậu tố
2.8 Minh họa chuyển trung tố sang hậu tố
2.9 Ứng dụng 3: tính giá trị biểu thức hậu tố
2.10 Ứng dụng 4: khử đệ quy bằng ngăn xếp

3. Hàng đợi (Queue)

3.1 Hàng đợi và vấn đề "tràn giả" khi cài bằng mảng
3.2 Cài đặt hàng đợi bằng mảng vòng
3.3 Cài đặt hàng đợi bằng danh sách liên kết
3.4 Ứng dụng hàng đợi: mô phỏng hàng chờ

4. Tổng kết và thực hành

4.1 Tóm tắt và bài thực hành buổi 9
4.2 Câu hỏi và bài tập

1.1 Khái niệm danh sách hạn chế

Định nghĩa

Danh sách hạn chế là danh sách mà các thao tác **thêm** và **hủy** phần tử chỉ được thực hiện ở **đầu** hoặc **cuối** danh sách (không được thao tác ở giữa). Hai loại quan trọng: **ngăn xếp (Stack)** và **hàng đợi (Queue)**.

Đặc điểm	Ngăn xếp (Stack)	Hàng đợi (Queue)
Nguyên tắc	**LIFO** – vào sau, ra trước	**FIFO** – vào trước, ra trước
Thêm	Ở **đỉnh** (Push)	Ở **cuối** (Add / EnQueue)
Lấy ra	Ở **đỉnh** (Pop)	Ở **đầu** (Get / DeQueue)
Hình ảnh	Chồng đĩa, chồng sách	Hàng người mua vé

Ví dụ 9.1 – Cùng thêm 1, 2, 3 rồi lấy ra 3 lần

- **Stack:** Push 1, 2, 3 ⇒ đỉnh là 3 ⇒ Pop lần lượt được **3, 2, 1**.
- **Queue:** Add 1, 2, 3 ⇒ Get lần lượt được **1, 2, 3**.

- **Ứng dụng của Stack:** lời gọi hàm và đệ quy, Undo, kiểm tra dấu ngoặc, tính biểu thức, duyệt sâu (DFS), quay lui.
- **Ứng dụng của Queue:** hàng đợi in, bộ đệm bàn phím/mạng, lập lịch CPU, mô phỏng hàng chờ, duyệt rộng (BFS).

- Stack và Queue là các **kiểu dữ liệu trừu tượng**: định nghĩa qua phép toán; cài đặt được bằng **danh sách đặc (mảng)** hoặc **danh sách liên kết**.

2.1 Các phép toán trên ngăn xếp

Phép toán	Ý nghĩa	Độ phức tạp
Initialize	Tạo ngăn xếp rỗng	O(1)
IsEmpty / IsFull	Kiểm tra rỗng / đầy	O(1)
Push(X)	Thêm X vào **đỉnh**	O(1)
Pop()	Lấy (và xóa) phần tử ở **đỉnh**	O(1)
Top()	Xem phần tử ở đỉnh (không xóa)	O(1)

- **Lỗi cần kiểm tra:** Push khi stack **đầy** (tràn – overflow); Pop/Top khi stack **rỗng** (underflow).
- Chỉ truy cập được phần tử ở đỉnh; muốn lấy phần tử ở dưới phải Pop hết các phần tử phía trên.

Ví dụ 9.2 – Chuỗi thao tác

Thao tác	M[0..Top]	Top	Giá trị lấy ra
Init		−1	
Push 5	5	0	
Push 8	5 8	1	
Push 3	5 8 3	2	
Pop	5 8	1	3
Push 7	5 8 7	2	
Pop	5 8	1	7
Pop	5	0	8

2.2 Cài đặt ngăn xếp bằng mảng

```
#define MaxLen 100
typedef struct {
   T   M[MaxLen];
   int Top;              // chỉ số phần tử đỉnh
} Stack;
void S_Init(Stack &S)     { S.Top = -1; }
int  S_IsEmpty(Stack S)   { return S.Top == -1; }
int  S_IsFull(Stack S)    { return S.Top == MaxLen - 1; }
int  S_Push(Stack &S, T X)
{  if (S_IsFull(S)) return 0;      // tràn stack
   S.M[++S.Top] = X;
   return 1;
}
int  S_Pop(Stack &S, T &X)
{  if (S_IsEmpty(S)) return 0;     // stack rỗng
   X = S.M[S.Top--];
   return 1;
}
T    S_Top(Stack S)       { return S.M[S.Top]; }
```

- **Biểu diễn:** mảng M chứa các phần tử từ **đáy** (M[0]) tới **đỉnh** (M[Top]); Top = −1 nghĩa là rỗng.
- `S.M[++S.Top] = X`: tăng Top **trước** rồi gán ⇒ X nằm ở ô trống kế tiếp.
- `X = S.M[S.Top--]`: lấy giá trị ở đỉnh **rồi** giảm Top. Phần tử cũ vẫn nằm trong mảng nhưng không còn thuộc stack.
- Tham số `Stack &S` trong Push, Pop vì hàm thay đổi stack; IsEmpty chỉ đọc nên truyền giá trị (nên dùng `const Stack &S` để tránh sao chép cả mảng).

Ưu, nhược điểm

Mọi phép toán **O(1)**, cài đặt đơn giản, nhanh. Nhược điểm: giới hạn **MaxLen** – chọn quá nhỏ thì tràn, quá lớn thì lãng phí.

2.3 Cài đặt ngăn xếp bằng danh sách liên kết đơn

- Đỉnh stack = **nút đầu** DSLK ⇒ Push = thêm đầu, Pop = hủy đầu (đều O(1), buổi 7).

```
typedef struct SNode {
   T Key;
   SNode *NextNode;
} SNode;
typedef SNode *LStack;        // con trỏ tới đỉnh
void LS_Init(LStack &S)       { S = NULL; }
int  LS_IsEmpty(LStack S)     { return S == NULL; }
int  LS_Push(LStack &S, T X)
{  SNode *p = new SNode;
   if (p == NULL) return 0;
   p->Key = X;
   p->NextNode = S;           // thêm đầu
   S = p;
   return 1;
}
int  LS_Pop(LStack &S, T &X)
{  if (S == NULL) return 0;
   SNode *p = S;
   X = p->Key;
   S = S->NextNode;           // hủy đầu
   delete p;
   return 1;
}
```

Tiêu chí	Stack mảng	Stack DSLK
Kích thước	Cố định MaxLen	Không giới hạn
Bộ nhớ mỗi phần tử	sizeof(T)	sizeof(T) + con trỏ
Tốc độ	Nhanh (không cấp phát)	Chậm hơn (new/delete)
Lỗi có thể gặp	Tràn (overflow)	Hết bộ nhớ

Vì sao không dùng nút cuối làm đỉnh?

Với DSLK đơn, Pop ở cuối cần tìm nút đứng trước nút cuối ⇒ **O(N)**. Chọn đầu danh sách làm đỉnh để mọi thao tác O(1).

- **Stack của hệ thống:** mỗi lời gọi hàm đẩy (Push) một khung chứa tham số, biến cục bộ, địa chỉ trở về; kết thúc hàm thì Pop ⇒ đệ quy quá sâu gây lỗi **stack overflow**.

2.4 Ứng dụng 1: đảo chuỗi và kiểm tra dấu ngoặc

- **Đảo chuỗi:** Push lần lượt các ký tự, rồi Pop hết ra.

```
void DaoChuoi(char *s)
{  Stack S;  S_Init(S);  char c;
   for (int i = 0; s[i]; i++) S_Push(S, s[i]);
   int i = 0;
   while (S_Pop(S, c)) s[i++] = c;
}
```

- "STACK" ⇒ Push S, T, A, C, K ⇒ Pop: **"KCATS"**.
- **Kiểm tra dấu ngoặc cân bằng** – ý tưởng:
  - Gặp ngoặc **mở** ⇒ Push.
  - Gặp ngoặc **đóng** ⇒ Pop và kiểm tra **cùng loại**; nếu stack rỗng ⇒ thiếu ngoặc mở.
  - Cuối chuỗi stack phải **rỗng** (không dư ngoặc mở).

```
int KiemTraNgoac(const char *s)
{  Stack S;  S_Init(S);  char x;
   for (int i = 0; s[i]; i++)
   {  char c = s[i];
      if (c == '(' || c == '[' || c == '{')
         S_Push(S, c);
      else if (c == ')' || c == ']' || c == '}')
      {  if (!S_Pop(S, x)) return 0;
         if ((c == ')' && x != '(') ||
             (c == ']' && x != '[') ||
             (c == '}' && x != '{'))
            return 0;
      }
   }
   return S_IsEmpty(S);
}
```

2.5 Minh họa kiểm tra dấu ngoặc

Ví dụ 9.3 – Kiểm tra "{a[(b+c)*d]}"

Ký tự	Thao tác	Stack
{	Push	{
a	Bỏ qua	{
[	Push	{ [
(	Push	{ [(
b + c	Bỏ qua	{ [(
)	Pop "(" – khớp	{ [
* d	Bỏ qua	{ [
]	Pop "[" – khớp	{
}	Pop "{" – khớp	rỗng
Kết thúc	Stack rỗng	**Hợp lệ**

Chuỗi	Kết quả	Lý do
(a+b]	Không hợp lệ	"]" gặp "(" – sai loại
((a)	Không hợp lệ	Cuối chuỗi stack còn "("
a)(	Không hợp lệ	")" khi stack rỗng
([)]	Không hợp lệ	")" gặp "[" – lồng sai

- **Vì sao đếm số ngoặc mở/đóng là không đủ?** "([)]" có 2 mở, 2 đóng nhưng sai; chỉ stack mới kiểm tra được thứ tự **lồng nhau**.
- **Ứng dụng:** trình biên dịch kiểm tra cú pháp `{ }` trong C/C++, kiểm tra thẻ HTML/XML mở – đóng.

2.6 Ứng dụng 2: chuyển biểu thức trung tố sang hậu tố

- **Trung tố** (infix): toán tử ở giữa – `A + B * C`, cần ngoặc và độ ưu tiên. **Hậu tố** (postfix, ký pháp Ba Lan ngược): toán tử đứng **sau** toán hạng – `A B C * +`, **không cần ngoặc**, tính được bằng một lần duyệt.
- **Thuật toán (dùng stack chứa toán tử):** duyệt từ trái sang phải:
  - **Toán hạng** ⇒ đưa ngay ra kết quả.
  - **"("** ⇒ Push. **")"** ⇒ Pop đưa ra kết quả cho đến khi gặp "(" (bỏ cặp ngoặc).
  - **Toán tử** op ⇒ Pop đưa ra các toán tử ở đỉnh có độ ưu tiên **≥** op, rồi Push op. (Ưu tiên: * / = 2; + − = 1; "(" = 0.)
  - Hết biểu thức ⇒ Pop đưa ra toàn bộ stack.

Trung tố	Hậu tố
A + B	A B +
A + B * C	A B C * +
(A + B) * C	A B + C *
A − B − C	A B − C − (kết hợp trái)

- Điều kiện "≥" (không phải ">") đảm bảo toán tử cùng ưu tiên được tính **từ trái sang phải**: A − B − C = (A − B) − C.

2.7 Cài đặt chuyển trung tố sang hậu tố

```
int UuTien(char op)
{  if (op == '*' || op == '/') return 2;
   if (op == '+' || op == '-') return 1;
   return 0;                    // '('
}
void TrungToSangHauTo(const char *in, char *out)
{  Stack S;  S_Init(S);  int k = 0;  char x;
   for (int i = 0; in[i]; i++)
   {  char c = in[i];
      if (c == ' ') continue;
      if (isalnum(c)) out[k++] = c;
      else if (c == '(') S_Push(S, c);
      else if (c == ')')
      {  while (S_Pop(S, x) && x != '(') out[k++] = x;  }
      else
      {  while (!S_IsEmpty(S) && UuTien(S_Top(S)) >=
UuTien(c))
         {  S_Pop(S, x);  out[k++] = x;  }
         S_Push(S, c);
      }
   }
   while (S_Pop(S, x)) out[k++] = x;
   out[k] = '\0';
}
```

- **UuTien:** "(" có ưu tiên thấp nhất (0) nên không bao giờ bị Pop bởi toán tử – chỉ bị Pop khi gặp ")".
- **isalnum(c):** chữ cái hoặc chữ số ⇒ toán hạng (bản đơn giản: mỗi toán hạng 1 ký tự).
- `while (S_Pop(S, x) && x != '(')`: Pop đưa ra cho đến khi gặp "(" – chính "(" bị Pop nhưng **không** đưa ra.
- Mỗi ký hiệu được Push và Pop **tối đa một lần** ⇒ độ phức tạp **O(n)**, n là độ dài biểu thức.

Mở rộng

Toán hạng nhiều chữ số: đọc liên tiếp các chữ số và chèn **dấu cách** sau mỗi toán hạng ở chuỗi kết quả; thêm toán tử lũy thừa "^" (ưu tiên 3, **kết hợp phải** ⇒ dùng ">" thay cho "≥").

2.8 Minh họa chuyển trung tố sang hậu tố

Ví dụ 9.4 – Chuyển A + B * (C − D) / E

Ký tự	Thao tác	Stack (đáy → đỉnh)	Kết quả
A	Đưa ra		A
+	Push	+	A
B	Đưa ra	+	A B
*	Đỉnh "+" ưu tiên 1 < 2 ⇒ Push	+ *	A B
(	Push	+ * (	A B
C	Đưa ra	+ * (	A B C
−	Đỉnh "(" ⇒ Push	+ * (−	A B C
D	Đưa ra	+ * (−	A B C D
)	Pop "−" ra, bỏ "("	+ *	A B C D −
/	Pop "*" (2 ≥ 2), dừng ở "+", Push "/"	+ /	A B C D − *
E	Đưa ra	+ /	A B C D − * E
Hết	Pop "/", "+"		**A B C D − * E / +**

2.9 Ứng dụng 3: tính giá trị biểu thức hậu tố

- **Thuật toán (stack chứa toán hạng):** duyệt biểu thức hậu tố từ trái sang phải:
  - **Toán hạng** ⇒ Push.
  - **Toán tử** op ⇒ Pop **b** (lấy ra trước), Pop **a**, tính **a op b**, Push kết quả.
  - Hết biểu thức ⇒ giá trị là phần tử duy nhất còn trong stack.

```
double TinhHauTo(const char *s)    // toán hạng 1 chữ số
{  double St[MaxLen];  int Top = -1;
   for (int i = 0; s[i]; i++)
   {  char c = s[i];
      if (isdigit(c)) St[++Top] = c - '0';
      else
      {  double b = St[Top--], a = St[Top--];
         switch (c)
         {  case '+': St[++Top] = a + b; break;
            case '-': St[++Top] = a - b; break;
            case '*': St[++Top] = a * b; break;
            case '/': St[++Top] = a / b; break;
         }
      }
   }
   return St[Top];
}
```

Ví dụ 9.5 – (3 + 4) * 5 − 6 / 2 ⇒ hậu tố 3 4 + 5 * 6 2 / −

Ký hiệu	Thao tác	Stack
3	Push	3
4	Push	3 4
+	3 + 4 = 7	7
5	Push	7 5
*	7 * 5 = 35	35
6	Push	35 6
2	Push	35 6 2
/	6 / 2 = 3	35 3
−	35 − 3 = 32	**32**

- **Thứ tự a, b quan trọng** với − và /: Pop lần đầu được **b** (toán hạng phải).
- Biểu thức lỗi (thiếu toán hạng) ⇒ Pop khi stack rỗng ⇒ phải kiểm tra trong chương trình thực tế.

2.10 Ứng dụng 4: khử đệ quy bằng ngăn xếp

- Mọi thuật toán đệ quy đều có thể viết **không đệ quy** bằng cách tự quản lý một stack lưu các "việc còn phải làm" – tránh tràn stack hệ thống, chủ động bộ nhớ.

Ví dụ 9.6 – QuickSort không đệ quy

```
void QuickSortNR(T M[], int N)
{  int StL[MaxLen], StR[MaxLen], Top = -1;
   StL[++Top] = 0;  StR[Top] = N - 1;    // Push(0, N-1)
   while (Top >= 0)
   {  int Left = StL[Top], Right = StR[Top--];  // Pop
      T X = M[(Left + Right)/2];
      int i = Left, j = Right;
      do
      {  while (M[i] < X) i++;
         while (M[j] > X) j--;
         if (i <= j) { Swap(M[i], M[j]); i++; j--; }
      } while (i <= j);
      if (Left < j)  { StL[++Top] = Left; StR[Top] = j; }
      if (i < Right) { StL[++Top] = i; StR[Top] = Right; }
   }
}
```

Ví dụ 9.7 – Đổi số thập phân sang nhị phân

```
void DoiNhiPhan(int n)
{  Stack S;  S_Init(S);  char c;
   while (n > 0)
   {  S_Push(S, '0' + n % 2);   // số dư
      n /= 2;
   }
   while (S_Pop(S, c)) printf("%c", c);
}
```

- n = 13: dư lần lượt 1, 0, 1, 1 (Push) ⇒ Pop ra **1101**.
- Số dư sinh ra theo thứ tự **ngược** với kết quả cần in ⇒ đúng bản chất LIFO.

- **Quy tắc chung:** đầu vòng lặp Pop một "bài toán con"; thay mỗi lời gọi đệ quy bằng một lần Push tham số tương ứng.

3.1 Hàng đợi và vấn đề "tràn giả" khi cài bằng mảng

Phép toán	Ý nghĩa
Initialize, IsEmpty, IsFull	Khởi tạo, kiểm tra rỗng/đầy
Add(X) (EnQueue)	Thêm X vào **cuối** hàng đợi (Rear)
Get() (DeQueue)	Lấy ra phần tử ở **đầu** hàng đợi (Front)

- **Cài bằng mảng thường:** Add tăng Rear, Get tăng Front. Cả hai chỉ số **chỉ tăng** ⇒ vùng đầu mảng đã Get không được dùng lại.

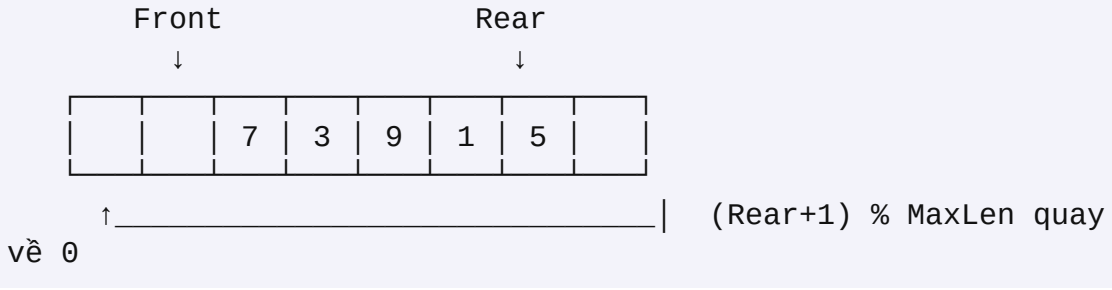
Ví dụ 9.7a – Tràn giả, MaxLen = 4

Add 10, 20, 30, 40 (Rear = 3) ⇒ Get 2 lần (Front = 2) ⇒ Add 50: Rear = 4 vượt mảng ⇒ báo **đầy** dù M[0], M[1] đang trống.

- **Cách khắc phục 1 – dời:** mỗi lần Get dời toàn bộ phần tử lên đầu ⇒ Get thành **O(N)** – không tốt.
- **Cách khắc phục 2 – mảng vòng:** xem mảng như vòng tròn, khi chỉ số vượt cuối thì **quay về 0**:

```
i = (i + 1) % MaxLen;
```

- **Phân biệt rỗng và đầy:** trong mảng vòng, Front và Rear có cùng quan hệ ở cả hai trạng thái ⇒ dùng thêm biến **Count** (số phần tử), hoặc chừa trống một ô.



3.2 Cài đặt hàng đợi bằng mảng vòng

```
typedef struct {
   T   M[MaxLen];
   int Front, Rear, Count;
} Queue;
void Q_Init(Queue &Q)    { Q.Front = 0; Q.Rear = -1; Q.Count
= 0; }
int  Q_IsEmpty(Queue Q)  { return Q.Count == 0; }
int  Q_IsFull(Queue Q)   { return Q.Count == MaxLen; }
int  Q_Add(Queue &Q, T X)
{  if (Q_IsFull(Q)) return 0;
   Q.Rear = (Q.Rear + 1) % MaxLen;
   Q.M[Q.Rear] = X;  Q.Count++;
   return 1;
}
int  Q_Get(Queue &Q, T &X)
{  if (Q_IsEmpty(Q)) return 0;
   X = Q.M[Q.Front];
   Q.Front = (Q.Front + 1) % MaxLen;  Q.Count--;
   return 1;
}
```

Ví dụ 9.8 – MaxLen = 4

Thao tác	Front	Rear	Count	M[0..3]
Init	0	−1	0	
Add 10, 20, 30	0	2	3	10 20 30 _
Get ⇒ 10	1	2	2	· 20 30 _
Get ⇒ 20	2	2	1	· · 30 _
Add 40	2	3	2	· · 30 40
Add 50	2	**0**	3	**50** · 30 40
Add 60	2	1	4	50 60 30 40 (đầy)
Get ⇒ 30	3	1	3	50 60 · 40

- Add 50: Rear = (3 + 1) % 4 = **0** – quay vòng tận dụng ô trống đầu mảng.

3.3 Cài đặt hàng đợi bằng danh sách liên kết

- Front = **nút đầu**, Rear = **nút cuối** DSLK đơn (buổi 7, trang 3.2): Add = thêm cuối O(1), Get = hủy đầu O(1).

```
typedef struct QNode { T Key; QNode *NextNode; } QNode;
typedef struct { QNode *Front, *Rear; } LQueue;
void LQ_Init(LQueue &Q) { Q.Front = Q.Rear = NULL; }
int  LQ_Add(LQueue &Q, T X)
{  QNode *p = new QNode;
   if (p == NULL) return 0;
   p->Key = X;  p->NextNode = NULL;
   if (Q.Rear == NULL) Q.Front = Q.Rear = p;
   else { Q.Rear->NextNode = p;  Q.Rear = p; }
   return 1;
}
int  LQ_Get(LQueue &Q, T &X)
{  if (Q.Front == NULL) return 0;
   QNode *p = Q.Front;
   X = p->Key;
   Q.Front = p->NextNode;
   if (Q.Front == NULL) Q.Rear = NULL;   // hàng đợi rỗng
   delete p;
   return 1;
}
```

Tiêu chí	Mảng vòng	DSLK
Kích thước	Cố định	Không giới hạn
Add / Get	O(1)	O(1)
Điểm cần chú ý	Phép % để quay vòng; phân biệt rỗng/đầy (dùng Count)	Cập nhật Rear khi hàng đợi trở thành rỗng

Vì sao Front ở đầu, Rear ở cuối DSLK (không ngược lại)?

Hủy ở **đầu** DSLK đơn O(1) nhưng hủy ở **cuối** O(N). Get (hủy) phải O(1) ⇒ đặt Front ở đầu danh sách.

- **Hàng đợi hai đầu (Deque)** – thêm/hủy ở cả hai đầu: cài bằng **DSLK kép** (buổi 8) hoặc mảng vòng.
- **Hàng đợi ưu tiên** – Get lấy phần tử ưu tiên cao nhất: cài bằng **heap** (buổi 4).

3.4 Ứng dụng hàng đợi: mô phỏng hàng chờ

Ví dụ 9.9 – Quầy giao dịch một nhân viên: khách đến ở các thời điểm 0, 1, 2, 6, 7; mỗi khách phục vụ 3 phút. Tính thời gian chờ trung bình.

```
typedef struct { int Ma, Den; } Khach;
// Q là hàng đợi Khach (mảng vòng hoặc DSLK)
int ThoiGian = 0, TongCho = 0, SoKhach = 0;
int RanhLuc = 0;                  // thời điểm quầy rảnh
for (int t = 0; t <= 30; t++)     // mỗi bước = 1 phút
{  // khách đến tại t ⇒ Add vào hàng đợi
   while (CoKhachDen(t, k)) Q_Add(Q, k);
   // quầy rảnh và có khách đợi ⇒ Get khách đầu hàng
   if (t >= RanhLuc && !Q_IsEmpty(Q))
   {  Q_Get(Q, k);
      TongCho += t - k.Den;  SoKhach++;
      RanhLuc = t + 3;
   }
}
printf("Cho TB: %.2f\n", (float)TongCho / SoKhach);
```

Khách	Đến	Bắt đầu phục vụ	Chờ	Hàng đợi khi khách đến
1	0	0	0	rỗng
2	1	3	2	(2)
3	2	6	4	(2, 3)
4	6	9	3	(3, 4) → 3 vào quầy ngay tại t = 6
5	7	12	5	(4, 5)

- Tổng thời gian chờ = 0 + 2 + 4 + 3 + 5 = **14** ⇒ trung bình **2,8 phút**.
- Khách được phục vụ **đúng thứ tự đến** ⇒ FIFO.
- Mở rộng: 2 quầy, thời gian phục vụ ngẫu nhiên ⇒ đánh giá cần bao nhiêu quầy để thời gian chờ < 5 phút.

4.1 Tóm tắt và bài thực hành buổi 9

Cấu trúc	Nguyên tắc	Thao tác chính	Cài đặt mảng	Cài đặt DSLK	Ứng dụng tiêu biểu
Stack	LIFO	Push, Pop, Top	M[], Top	Đỉnh = nút đầu	Ngoặc, biểu thức, đệ quy, Undo
Queue	FIFO	Add, Get	Mảng vòng Front, Rear, Count	Front = đầu, Rear = cuối	Hàng chờ, bộ đệm, BFS

Thực hành – Tính giá trị biểu thức hậu tố bằng Stack

- 1) Cài đặt Stack bằng mảng và bằng DSLK (hai thư viện có **cùng tên hàm**) – chương trình chính chạy được với cả hai.
- 2) Viết hàm `TrungToSangHauTo` (toán hạng là chữ cái hoặc số một chữ số; toán tử + − * /; có ngoặc) và in **bảng mô phỏng** từng bước như ví dụ 9.4.
- 3) Viết hàm `TinhHauTo` cho số **nhiều chữ số** (các phần tử cách nhau bởi dấu cách, ví dụ "12 3 + 4 *" ⇒ 60).
- 4) Kết hợp: nhập biểu thức trung tố ⇒ kiểm tra ngoặc ⇒ chuyển hậu tố ⇒ tính giá trị. Kiểm thử: `(3+4)*5-6/2` = 32; `2*(3+4*(5-1))` = 38; `8/(4-4)` ⇒ báo lỗi chia cho 0.
- 5) (Mở rộng) Cài đặt hàng đợi mảng vòng và mô phỏng ví dụ 9.9 với dữ liệu nhập từ bàn phím.

4.2 Câu hỏi và bài tập

Câu hỏi

- 1) Phân biệt LIFO và FIFO; cho 3 ví dụ thực tế cho mỗi loại.
- 2) Vì sao cài đặt hàng đợi bằng mảng nên dùng mảng vòng? Làm sao phân biệt hàng đợi rỗng và đầy?
- 3) Khi cài Stack bằng DSLK đơn, nên chọn đầu hay cuối danh sách làm đỉnh? Vì sao?

Bài tập (có đáp số)

- 1) Chuyển sang hậu tố và tính giá trị: `(8 - 2) * (3 + 1) / 4`. → **8 2 − 3 1 + * 4 /** = 6 × 4 / 4 = **6**.
- 2) Cho dãy thao tác trên stack rỗng: Push 1, Push 2, Pop, Push 3, Push 4, Pop, Pop, Push 5. Các giá trị lấy ra theo thứ tự? Stack còn lại? → Lấy ra **2, 4, 3**; còn **1 5** (đỉnh là 5).
- 3) Dùng hai stack để cài đặt một hàng đợi (Add vào stack 1; Get lấy từ stack 2, khi stack 2 rỗng thì đổ toàn bộ stack 1 sang). Chứng minh mỗi phần tử chỉ được chuyển tối đa một lần.
- 4) Viết chương trình kiểm tra một chuỗi có phải **palindrome** không bằng cách dùng đồng thời một stack và một queue.